

A1 - Vector Space Models

Yauheni Zviazdou

April 19, 2026

1 Introduction

This report details the implementation and evaluation of an experimental Information Retrieval system based on the Vector Space Model (VSM). The system is designed to rank documents according to their relevance to specific topics using an inverted index. It supports both English and Czech test collections and experiments with various text preprocessing techniques, term/document weighting schemes, and query expansion methods.

2 System Architecture Overview

The system is implemented in Python and operates entirely via the command line.

2.1 System's Possibilities

The system supports the following features:

- **Text preprocessing techniques:** lemmatization (using MorphoDiTa tagger [3]), case folding, Part-of-Speech based stopping, and numbers normalization.
- **Document/Query weighting schemes:** (nnn, nnc, ntn, ntc, lnn, lnc, ltn, ltc) for both documents and queries.
- **Query expansion:** Thesaurus-based expansion using synonyms from WordNet
 - For English, the system uses the NLTK WordNet [1] interface to retrieve synonyms.
 - For Czech, the system uses the Czech WordNet 1.9 PDT [2] to retrieve synonyms.
- **Multiprocessing:** Parallel processing capabilities to speed up processing of large document collections.

2.2 Project structure

The project codebase is systematically structured into the following core components and configuration files:

- **run**: Serves as the primary executable entry point for the system. It handles command-line argument parsing, environment initialization, and orchestrates the end-to-end retrieval pipeline.
- **main.py**: Encapsulates the core algorithmic logic. It is responsible for constructing the inverted index, applying the selected term-weighting schemes, computing vector space similarities, and ranking documents against the provided queries.
- **parser.py**: Manages the ingestion and preprocessing of structured input data. This includes parsing TREC-formatted topic files and extracting relevant textual content from the SGML-based English and Czech document collections.
- **Makefile**: Automates the project build process, streamlining environment setup and ensuring consistent execution.
- **process_wordnet_cs.py**: A specialized utility script that processes the Czech WordNet dataset to generate an optimized, custom JSON dictionary specifically utilized for thesaurus-based query expansion.
- **download_external.sh**: An automation script designated for acquiring requisite external assets, including the MorphoDiTa tagger models, document and query collections, linguistic databases, and **trec_eval** tool.
- **requirements.txt**: Defines the necessary Python runtime dependencies to guarantee a reproducible execution environment.
- **README.md**: Provides documentation, detailing system prerequisites, build instructions, and guidelines for reproducing the experimental runs.

The core operational workflow of the system is driven by two primary functions in **main.py**:

- **init_index(args)**: Manages the construction & saving of the inverted index from the document collection. It also includes built-in functionality to load a pre-computed index from persistent storage to expedite iterative experimentation.
- **retrieve_documents(args)**: Executes the query evaluation and retrieval phase. It computes the vector space similarities between the formulated queries and the indexed documents to generate a ranked list of relevant results.

3 Building and Execution Instructions

3.1 Building

Download the source code and navigate to the project directory. Then, run **make** to build the project.

Disclaimer : the build process primarily involves setting up the Python environment and downloading necessary external resources, rather than traditional compilation.

3.2 Execution

The system operates via the command line using the `run` script. The script accepts various arguments to specify the input files, output file, and configuration options for the retrieval process.

3.2.1 Example of use

```
./run -q topics.xml -d documents.lst -r run -o sample.res
```

3.3 Arguments and Descriptions

Argument	Description
-q	Required. A file including topics in the TREC format (.xml file).
-d	Required. A file including document filenames (.lst file).
-r	Required. A string identifying the experiment run.
-o	Required. The output file for retrieval results (.res file).
--top_k	Number of top documents to retrieve for each query. (Default: 1000)
--run_0	Flag to automatically apply the run-0 (baseline) setup.
--run_1	Flag to automatically apply the run-1 (constrained) setup.
--run_2	Flag to automatically apply the run-2 (unconstrained) setup.
--lemmatization	Flag to lemmatize tokens using the MorphoDiTa tagger.
--case_folding	Flag to apply case folding (lowercasing) to tokens.
--pos_stopping	Flag to apply Part-of-Speech based stopping.
--number_normalization	Flag to normalize all numeric digits to a standard <NUM> token.
--doc_weights	Weighting scheme for document terms. Possible values: (nnn, nnc, ntn, ntc, lnn, lnc, ltn, ltc).
--query_weights	Weighting scheme for query terms. Possible values: (nnn, nnc, ntn, ntc, lnn, lnc, ltn, ltc).
--use_query_description	Flag to expand the query using the topic <desc> and <narr> fields.
--thesaurus_n	Number of synonyms to retrieve from the WordNet/custom thesaurus.
--cpu_n	Number of CPU cores to allocate for parallel processing (-1 for all cores).
--save_index	Flag to save the constructed inverted index to disk for future runs.
--load_index	Flag to load a pre-computed inverted index from disk instead of constructing it from the document collection.

Table 1: Command-line arguments and descriptions for `run`

4 Experiments Setups

4.1 Run-0 Setup (Baseline)

The run-0 setup serves as a baseline for evaluating the performance of the retrieval system.

Settings for run-0:

- **tokenization:** whitespace+punctuation
- **class equivalence:** no
- **removing stopwords:** no
- **query construction:** all words from "title"
- **term weighting:** natural
- **document frequency weighting:** none
- **vector normalization:** cosine
- **similarity measurement:** cosine
- **relevance feedback:** none
- **query expansion:** none

This setup can be executed using the following command:

```
./run -q topics.xml -d documents.lst -r run-0 -o run-0.res --run-0
```

4.2 Run-1 Setup (Constrained)

The run-1 setup applies a series of preprocessing and weighting techniques to enhance the retrieval performance while adhering to the constraints specified in the assignment.

Settings for run-1:

- **tokenization:** whitespace+punctuation
- **class equivalence:** case folding, lemmatization (using MorphoDiTa tagger).
- **removing stopwords:** None.
- **query construction:** Topic titles only.
- **term weighting:** Logarithmic (1) for both documents and queries.
- **document frequency weighting:** IDF (t) for documents; None (n) for queries.
- **vector normalization:** None (n) for documents; Cosine normalization (c) for queries.
- **similarity measurement:** Cosine similarity.
- **relevance feedback:** None.
- **query expansion:** Thesaurus-based expansion (with scaling 0.2).

This setup can be executed using the following command:

```
./run -q topics.xml -d documents.lst -r run-1 -o run-1.res --run-1
```

5 Results and Evolution of Results

We conducted a series of experiments to evaluate the performance of the retrieval system under different configurations. The results are presented in terms of Mean Average Precision (MAP) for both English and Czech collections. Goal of the experiments was to iteratively improve the retrieval performance by applying various techniques.

5.1 Initial setup (Run-0)

The initial run-0 setup, which serves as a baseline, achieved MAP scores of 0.0411 for the English collection and 0.0596 for the Czech collection.

Setup	English (map)	Czech (map)
Run-0	0.0411	0.0596

Table 2: Initial setup (Run-0) results.

5.2 Documents and Queries weighting schemes

To optimize the Vector Space Model (VSM), we experimented with various term-weighting strategies. A complete weighting scheme consists of two parts: the document weighting and the query weighting.

Each weighting scheme is defined by a three-letter code representing three independent components: **Term Frequency**, **Document Frequency**, and **Normalization**. For this experiment, we implemented 2 specific possibilities for each component:

1. Term Frequency Weighting:

- **n** (natural): The raw term frequency is used without modification.
- **l** (logarithmic): The term frequency is dampened using a base-10 logarithm to reduce the disproportionate impact of highly frequent terms.

2. Document Frequency Weighting:

- **n** (none): No global weighting is applied. The multiplier is simply 1.
- **t** (inverse document frequency): Rare terms are rewarded, and common terms are penalized using IDF.

3. Vector Normalization:

- **n** (none): No normalization is applied to the final vector.
- **c** (cosine): The vector is scaled by its Euclidean length.

There is 64 possible combinations of weighting schemes for documents and queries, which are represented by three-letter codes for documents and queries, respectively.

We conducted a grid search across all 64 possible combinations. The Mean Average Precision (MAP) results for both the English and Czech test collections are summarized in Table 3.

Scheme	English	Czech	Scheme	English	Czech
nnn.nnn	0.0765	0.0366	lnn.nnn	0.1879	0.1278
nnn.nnc	0.0765	0.0366	lnn.nnc	0.1879	0.1278
nnn.ntn	0.1171	0.0595	lnn.ntn	0.2239	0.1626
nnn.ntc	0.1171	0.0595	lnn.ntc	0.2239	0.1626
nnn.lnn	0.0765	0.0366	lnn.lnn	0.1879	0.1278
nnn.lnc	0.0765	0.0366	lnn.lnc	0.1879	0.1278
nnn.ltn	0.1171	0.0595	lnn.ltn	0.2239	0.1626
nnn.ltc	0.1171	0.0595	lnn.ltc	0.2239	0.1626
nnc.nnn	0.0415	0.0594	lnc.nnn	0.0492	0.0692
nnc.nnc	0.0415	0.0594	lnc.nnc	0.0492	0.0692
nnc.ntn	0.0827	0.0939	lnc.ntn	0.0910	0.1087
nnc.ntc	0.0827	0.0939	lnc.ntc	0.0910	0.1087
nnc.lnn	0.0415	0.0594	lnc.lnn	0.0492	0.0692
nnc.lnc	0.0415	0.0594	lnc.lnc	0.0492	0.0692
nnc.ltn	0.0827	0.0939	lnc.ltn	0.0910	0.1087
nnc.ltc	0.0827	0.0939	lnc.ltc	0.0910	0.1087
ntn.nnn	0.1171	0.0595	ltn.nnn	0.2239	0.1626
ntn.nnc	0.1171	0.0595	ltn.nnc	0.2239	0.1626
ntn.ntn	0.1481	0.0913	ltn.ntn	0.1968	0.1556
ntn.ntc	0.1481	0.0913	ltn.ntc	0.1968	0.1556
ntn.lnn	0.1171	0.0595	ltn.lnn	0.2239	0.1626
ntn.lnc	0.1171	0.0595	ltn.lnc	0.2239	0.1626
ntn.ltn	0.1481	0.0913	ltn.ltn	0.1968	0.1556
ntn.ltc	0.1481	0.0913	ltn.ltc	0.1968	0.1556
ntc.nnn	0.0953	0.0869	ltc.nnn	0.0848	0.0975
ntc.nnc	0.0953	0.0869	ltc.nnc	0.0848	0.0975
ntc.ntn	0.1117	0.1292	ltc.ntn	0.1006	0.1178
ntc.ntc	0.1117	0.1292	ltc.ntc	0.1006	0.1178
ntc.lnn	0.0953	0.0869	ltc.lnn	0.0848	0.0975
ntc.lnc	0.0953	0.0869	ltc.lnc	0.0848	0.0975
ntc.ltn	0.1117	0.1292	ltc.ltn	0.1006	0.1178
ntc.ltc	0.1117	0.1292	ltc.ltc	0.1006	0.1178

Table 3: Weighting schemes for documents and queries. Results are shown in terms of MAP.

Scheme **ltn.lnc** was the best performing scheme for both English and Czech collections, achieving the highest MAP scores of 0.2239 and 0.1626, respectively. This scheme was chosen for the run-1 setup.

5.3 Lemmatization + POS-based stopping

Next step was to apply lemmatization, POS-based stopping and their combination. The results are shown in the Table 4.

Approach	English (map)	Czech (map)
pos_stopping	0.1865	0.1624
lemmatization	0.2527	0.3224
lemmatization + pos_stopping	0.2498	0.3177

Table 4: Lemmatization and POS-based stopping

Lemmatization significantly improved the MAP scores for both English and Czech collections, increasing them to 0.2527 and 0.3224, respectively. POS-based stopping alone did not improve the performance, probably POS-based stopping was too strict.

5.4 Query expansion

Next step was to apply thesaurus-based query expansion using synonyms from WordNet. We retrieved all synonyms and added weights of them to the query with additional scaling factor. To select the best scaling factor, we experimented with different values and found that 0.2 was the best performing one for both English and Czech collections, Fig. 1.

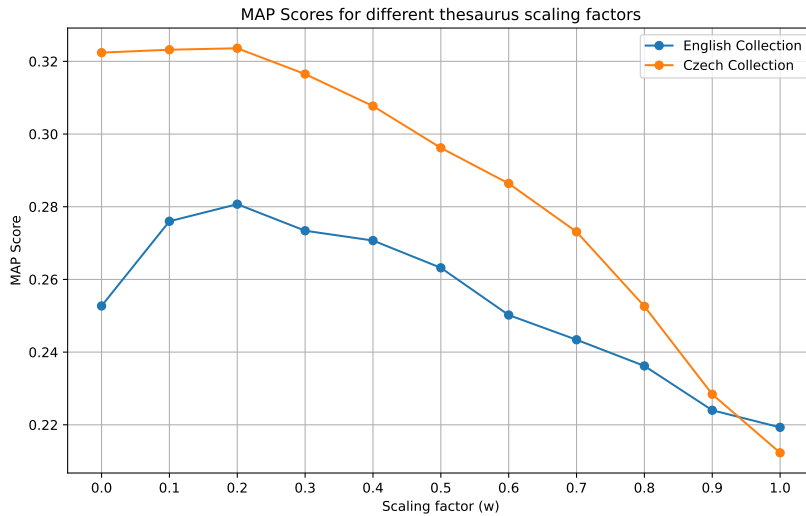


Figure 1: Thesaurus-based query expansion using synonyms from WordNet.

Detailed data of experiment are shown in the Table 5.

Scaling Factor	English (map)	Czech (map)
0.0	0.2527	0.3224
0.1	0.2760	0.3232
0.2	0.2807	0.3236
0.3	0.2734	0.3165
0.4	0.2707	0.3077
0.5	0.2632	0.2962
0.6	0.2502	0.2864
0.7	0.2434	0.2731
0.8	0.2362	0.2526
0.9	0.2240	0.2284
1.0	0.2193	0.2123

Table 5: Thesaurus-based query expansion

5.5 Final Results

Table 6 summarizes the overall performance leap from the initial baseline (Run-0) to the fully optimized constrained system (Run-1).

Approach	English (map)	Czech (map)
Run-0 (Baseline)	0.0411	0.0596
Run-1 (Constrained)	0.2807	0.3236

Table 6: Final Results

By systematically integrating the `ltn.lnc` weighting scheme, MorphoDiTa-based lemmatization, and appropriately scaled thesaurus expansion, the system achieved big improvements in retrieval performance for both English and Czech collections.

Run-1 is 6.8 times better than Run-0 for English collection and 5.4 times better for Czech collection in terms of MAP.

6 Summary

The implemented Information Retrieval system based on the Vector Space Model demonstrated significant improvements in retrieval performance through the application of lemmatization and thesaurus-based query expansion.

Lemmatization alone resulted in a substantial increase in MAP scores for both English and Czech collections, while the addition of POS-based stopping did not yield further benefits.

Thesaurus-based query expansion, when applied with an optimal scaling factor of 0.2, significantly improved the retrieval performance, further enhancing the MAP scores for both collections.

The final run-1 setup, which incorporated lemmatization and thesaurus-based query expansion, achieved the highest MAP scores 0.2807 (English) and 0.3236 (Czech).

References

- [1] Steven Bird and Edward Loper. NLTK: The natural language toolkit. In *Proceedings of the ACL Interactive Poster and Demonstration Sessions*, pages 214–217, Barcelona, Spain, July 2004. Association for Computational Linguistics.
- [2] Karel Pala, Tomáš Čapek, Barbora Zajíčková, Dita Bartůšková, Kateřina Kulková, Petra Hoffmannová, Eduard Bejček, Pavel Straňák, and Jan Hajič. Czech WordNet 1.9 PDT, 2011. LINDAT/CLARIAH-CZ digital library at the Institute of Formal and Applied Linguistics (ÚFAL).
- [3] Jana Straková, Milan Straka, and Jan Hajič. Open-Source Tools for Morphology, Lemmatization, POS Tagging and Named Entity Recognition. In *Proceedings of 52nd Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, pages 13–18, Baltimore, Maryland, June 2014. Association for Computational Linguistics.